

Metrikenbericht - KlausurCraft

Projekt: KlausurCraft

Datum der Auswertung: 01.04.2026

Messzeitpunkt: 13:05 CEST

1. Vorgehen

Die Kennzahlen wurden auf Basis des aktuellen Projektstands automatisch ermittelt. Für die Zählung der Codezeilen wurde `cloc` verwendet; weitere Kennzahlen (Pakete, Typen, Methoden, Imports) wurden über Skripte und Regex-Auswertungen bestimmt.

2. Kernmetriken der Implementierung

Kennzahl	Ergebnis
Java-Dateien	28
Lines of Code (Java, nur Code)	2393
Kommentarzeilen (Java)	190
Leerzeilen (Java)	500
Anzahl Pakete	8
Anzahl Klassen (<code>class</code>)	25

Zusätzlich ermittelte Typen: - Enums: 5 - Records: 3 - Interfaces: 0 - Typen gesamt (`class` + `enum` + `record` + `interface`): 33

3. Weitere Metriken

Kennzahl	Ergebnis	Anmerkung
Kommentaranteil	7,36 %	Verhältnis <code>Kommentar</code> / (<code>Kommentar</code> + <code>Code</code>)
Methoden (heuristisch gezählt)	136	Signaturbasierte Erkennung
Durchschnittliche Methodengröße	17,60 LoC	2393 / 136
Import-Anweisungen gesamt	215	Summe aller Java-Imports
Durchschnittliche Imports pro Typ	7,96	einfacher Indikator für Kopplung

Datei mit der höchsten Importzahl: - `src/main/java/simon/klausurcraft/ui/home/HomeGenerateFlow.java` mit 23 Imports.

4. Paketübersicht

1. `simon.klausurcraft.app`
2. `simon.klausurcraft.task`
3. `simon.klausurcraft.task.export`
4. `simon.klausurcraft.task.io`
5. `simon.klausurcraft.task.planning`
6. `simon.klausurcraft.ui`
7. `simon.klausurcraft.ui.components`
8. `simon.klausurcraft.ui.home`

5. Hinweise zu erweiterten IDE-Metriken

Für detailliertere Qualitätsmetriken wie Kohäsion (z. B. LCOM), zyklomatische Komplexität oder detaillierte Abhängigkeitsanalysen kann ergänzend ein IDE-Plugin eingesetzt werden (z. B. MetricsReloaded in IntelliJ oder CodeMR in Eclipse). Die hier dokumentierten Werte bilden die Basiskennzahlen des aktuellen Codebestands.

6. Reproduzierbarkeit

Die Auswertung kann mit folgendem Befehl erneut durchgeführt werden:

```
./scripts/collect-metrics.sh
```